

AI-Driven Hybrid-Hierarchical Engine Classifier

A study in what standard machine-learning models can and cannot do without domain knowledge.

99.8%

end-to-end accuracy

164,920-row held-out benchmark - 15 engine states - 4 sensor inputs

THIS REPORT ANSWERS, DIRECTLY:

- 1 Why a hierarchy of specialised models instead of one general model?
- 2 Why was the whole system rebuilt from scratch (v1.0 → v2.0)?
- 3 Why are some faults detected by hard-coded rules instead of by a model?

Contents

Abstract	3
1 Motivation	3
2 Problem Definition	3
3 Synthetic Data Generation	4
4 The Architectural Journey — Why the System Looks the Way It Does	5
5 The Final Architecture	8
6 Results	10
7 The Running System — From Models to Product	11
8 What the Project Revealed About the Limits of ML	12
9 Limitations and Future Work	12
Closing	13
References	13

HOW TO READ THIS REPORT

Sections 1–3 define the problem and the data. Section 4 is the heart of the document — the failures that produced every design decision. Sections 5–7 describe the final system and its results. Section 8 states what was actually learned.

— Abstract

This report documents the design, failure, and complete redesign of a system that classifies **15 engine states** from four time-series sensor inputs — Temperature, Pressure, RPM, and Vibration. The final system is a hierarchy that combines deterministic rules, Random Forest classifiers, and 1-D convolutional networks, reaching **99.8 % accuracy on a 164,920-row held-out benchmark**.

The accuracy is not the point of this report. The project began with a deliberately naïve premise — hand a set of generic ML models a problem and see how far they get on their own — and the answer became the actual finding: **on their own, not far**. Every architectural decision in the final system exists because a simpler approach was tried first and failed in a specific, instructive way. This report explains each of those failures and the decision it forced, because the decisions only make sense once you see what broke.

The short answer to all three cover questions is the same: **the models kept succeeding in isolation and failing in context**, and each failure revealed a task that a model should never have been given in the first place.

1 Motivation

There is a persistent gap between two kinds of engineer. The data scientist can build and train a model but often does not know the physics of the system the data came from. The mechanical engineer knows the physics but often treats machine learning as a black box. Neither side alone tends to ask the question that actually decides whether an ML system works: **what should the model be expected to learn, and what should it never be asked to learn at all?**

This project was a deliberate attempt to stand in that gap. The goal was never to build the most accurate engine classifier possible — a handful of **if** statements over RPM and temperature would classify most of these states trivially. The goal was to find out, first-hand, **where a generic ML model stops being able to do the job**, on a problem where the correct answer was already known from the physics, so that every failure would be legible instead of mysterious. Throughout the build, one rule was enforced deliberately: **no hints** — the models had to discover the patterns themselves, because handing them the answer would defeat the purpose of the experiment. (The single, deliberate exception is reported in Section 5.3.)

The biggest limitation of a machine-learning model is rarely the model itself. It is how the data is prepared, and what the model is expected to learn without enough context.

This is a documented phenomenon — sometimes called the *Clever Hans effect*, where a model appears intelligent but has latched onto a spurious pattern in the data. The value here was arriving at it experimentally, by watching it happen repeatedly on a system I understood well enough to diagnose.

2 Problem Definition

The system classifies an engine into one of **15 states**, in three families.

Operating states (12) — a healthy engine

Family	Sub-states
Engine Off	cold, warm
Engine Start	transient start event
Normal Load	idle, accelerating, decelerating
High Load	idle, accelerating, decelerating
Critical Load	idle, accelerating, decelerating

Fault states (3) — a sensor or data problem, not an engine condition

Fault	Meaning
Uncalibrated	a sensor reports a physically impossible value
Frozen Sensor	a sensor's value stops changing — stuck at a constant
Error Value	the sensor output is corrupted / non-numeric

Each state has a physical signature across the four variables, and encoding that signature is exactly where mechanical domain knowledge enters:

- **Engine Off** has RPM pinned at zero and pressure near atmospheric (~1 bar). *Cold* versus *warm* is separated only by temperature — and a still-hot engine ($\geq 95^\circ\text{C}$) shows slightly elevated vibration as it settles.
- **Load level** is fundamentally a function of RPM, with temperature, pressure, and vibration all rising as the engine works harder. These are **physically coupled** — you cannot have high RPM with cold temperature and low vibration in a real engine — and the data generators enforce that coupling rather than sampling each variable independently.
- **Manifold pressure above ~1.0 bar** is rare in a naturally aspirated engine, occurring only under ram-air or intake-resonance effects, so the plausible ceiling was set at ~1.02 bar — a decision from the physics, not from the data.
- **Behaviour** (idle / accelerating / decelerating) is defined entirely by the *change* in RPM between consecutive moments, independent of the absolute load.

Every later problem in the project traces back to the relationship between these physical definitions and how the data was structured **in time** — the thread running through Section 4.

3 Synthetic Data Generation

No real engine data was used. Every state was generated synthetically in Python with added noise. This gave full control over class definitions — and it also meant every “bad accuracy” result had to be interrogated, because a wrong number was as often a flaw in the *generator* as a limitation of the model. Several times, it was the generator. The exact ranges below are the ones used to train the final models; they are the reproducible core of the system.

Engine Off — 12,000 samples (8,000 / 2,000 / 2,000), shape (N, 1, 4)

Sub-state	Temperature	Vibration
Cold	-20 ... 50 °C	0.0001 ... 0.001
Warm	50 ... 120 °C	0.002 ... 0.005 if $T \geq 95^\circ\text{C}$, else 0.0001 ... 0.001

Both sub-states share pressure 0.98–1.02 bar and RPM 0.

Load states — 24,000 each (18,000 / 3,000 / 3,000), shape (N, 1, 4)

Load	Temperature	RPM	Pressure	Vibration
Normal	80–105 °C	800–3000	$0.6 + (\text{rpm}-700)/9000 \pm 0.03$	$0.05 + (\text{rpm}-700)/20000 \pm 0.02$
High	105–115 °C	2500–5500	0.9 ± 0.07	$0.10 + (\text{rpm}-3500)/20000 + (-0.04...0.05)$
Critical	115–145 °C	4500–9000	0.97 ± 0.05	$0.20 + (\text{rpm}-5000)/15000 + (-0.08...0.12)$

Pressure and vibration are **derived from RPM**, so the variables stay coupled and never drift out of range unless the load itself changes.

Engine Start — 12,000 samples, shape (N, 4, 4)

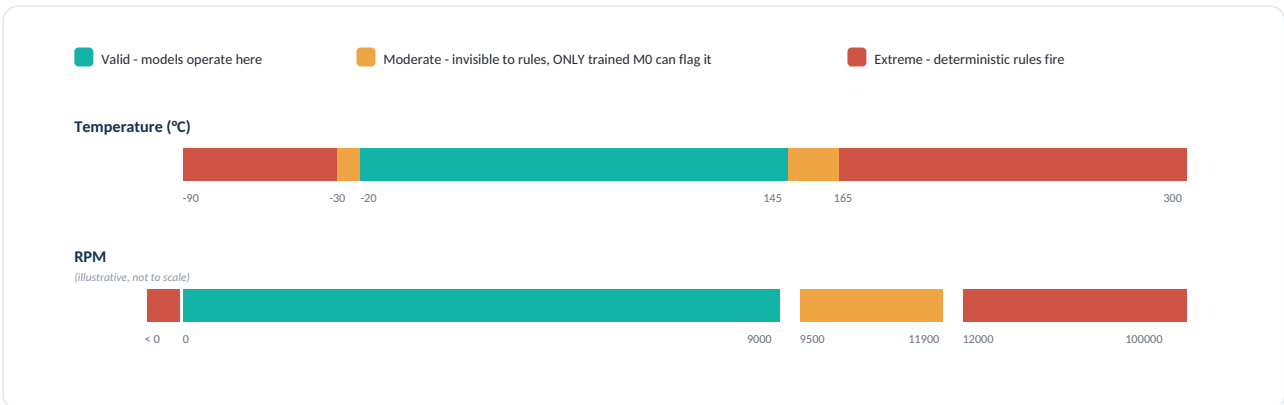
A **4-step window**, because a start is an event, not a snapshot. Each window opens with 1–3 *Off* steps, then transitions to *On*: temperature rises 3–7 °C per step; RPM leaps to 1500–3000 on the first firing step then settles down in stages; pressure dips (0.7–0.9) then recovers; vibration spikes (0.3–0.6) then settles.

Behaviour — 40,000 windows (36,000 / 2,000 / 2,000), shape (N, 2, 4)

A **2-step window**, the minimum needed to see a change in RPM. The class is defined purely by ΔRPM : accelerating +40...+300, decelerating -40...-300, steady -40...+40. The auxiliary variables are sampled across their **entire** operating range, so the model learns to read the RPM trend *without* leaning on any correlated variable.

Fault data — a two-tier design

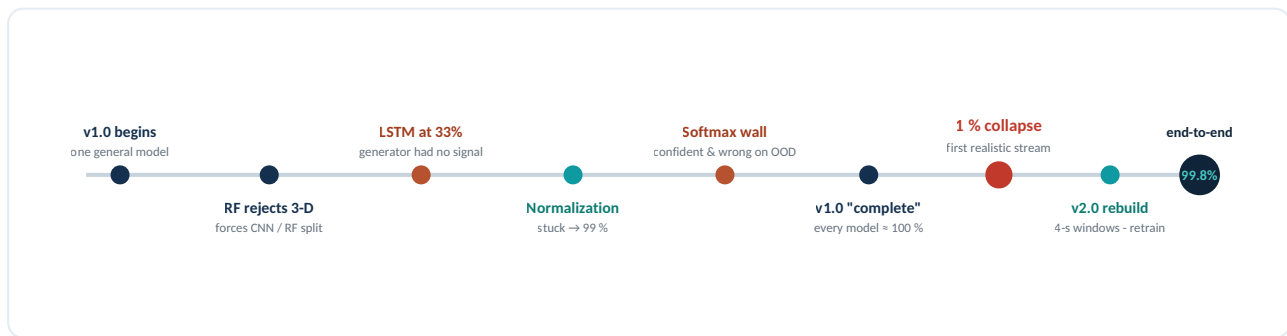
The uncalibrated-fault generator writes the eventual division of labour between rules and models *into the data itself*. It defines two zones per variable — an **extreme** zone that a deterministic rule catches with certainty, and a **moderate** zone: values above any valid operating range but *below* the rule's thresholds, therefore invisible to the rule. Training data is split 50/50 between the two zones, so the trained model learns to own exactly the region the rules cannot reach.



The rules see only the red zones. The amber zones exist so the trained model has a job the rules cannot do — Section 5.4 shows why this split was necessary at all. Pressure and Vibration follow the same three-zone pattern (e.g. Pressure: valid 0.58–1.16, moderate 1.30–1.45 / 0.05–0.45, extreme 1.5–6 / -6–0.01).

4 The Architectural Journey — Why the System Looks the Way It Does

The final architecture was not designed up front. It was arrived at by building a simpler system, watching it fail in specific ways, and rebuilding. Each failure below is the evidence for a decision in the final design.



4.1 Why not one general model?

The starting instinct was the data-scientist's instinct: assemble the data, hand it to one model, let it sort everything out. An early note of mine captures it exactly — *“it doesn't matter whether the data rises gradually or suddenly, the model interprets it, not me.”* The project went on to prove the precise opposite. Three things broke that plan:

- **A mechanical constraint forced the first split.** A Random Forest simply refuses 3-D time-series input of shape (N, 30, 4) — it requires a flatten that destroys the very time-ordering that load and behaviour depend on. That single constraint pushed the time-dependent parts of the system onto convolutional networks and left the Random Forests to the genuinely snapshot-like states. **The tool dictated part of the architecture.**
- **A single model overfit a trivial sub-problem.** The first start-vs-off model overfit badly — 93 % train, 68 % validation — on a task a one-line `if` solves perfectly. A general model brings no free lunch: it can *fail at the easy part* while you worry about the hard part.
- **Softmax models cannot say “I don't know.”** Fed data from a class it had never seen, a router returned a *confident* wrong answer (> 0.95), not a low-confidence one. Softmax spreads probability over the classes it *knows* and forces any out-of-distribution input toward the nearest one. The consequence is large: fault detection can never be a “flag it if confidence is low” threshold, because confidence does not drop for the unknown. The unknown state had to become something the system **decides deliberately**, not something it infers from hesitation.

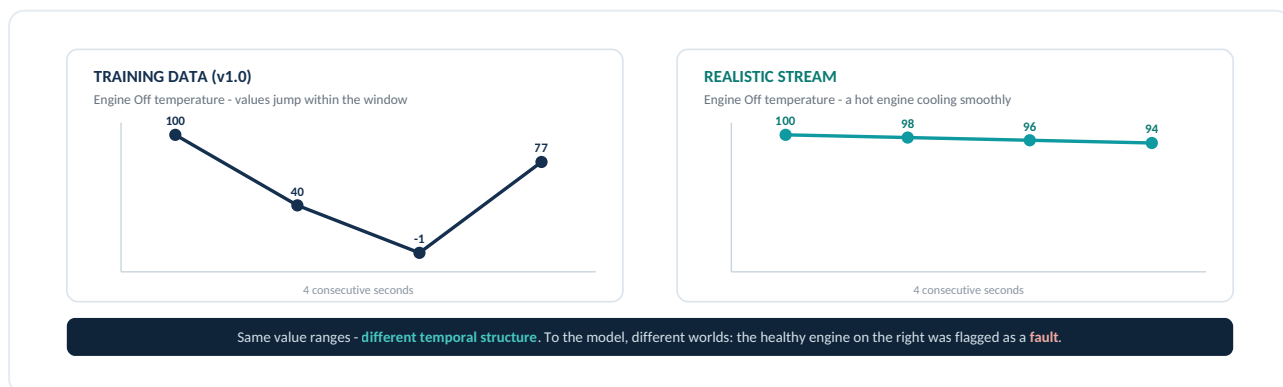
Together these pushed the design away from “one model” and toward a **hierarchy where each stage handles the kind of pattern it is actually suited for** — a snapshot for a Random Forest, a short temporal window for a CNN, a deterministic condition for a rule.

4.2 Why the whole system was rebuilt (v1.0 → v2.0)

This is the most important part of the project, and the reason is precise. By the end of v1.0, **every model worked in isolation**. Trained and tested on its own dataset, each scored at or near 100 %. The system was ~98 % built — data, models, fusion logic, evaluation all in place. The only step left was to run it on a realistic, continuous stream — and on the way there, the training datasets had to be **merged into one continuous flow** for the first time, the way a real engine would produce them.

IT COLLAPSED TO 1 % ACCURACY

The cause is the entire thesis in one concrete failure:



The models had been trained on Engine Off windows where the values, though in the correct ranges, **jumped around within the window**. But a realistic Engine Off does not jump; it **varies smoothly**. The **value ranges were identical**. The **temporal structure was completely different**. To a human, both are obviously an engine that is off. To the model, the smooth version was unlike anything in its training data, so it flagged a perfectly healthy idle engine as a *fault*.

This is why the failure only appeared at integration: each model was validated on data with the *same jumpy structure it was trained on*, so each looked perfect. Only when real, smoothly-varying data flowed through the assembled system did the mismatch surface. **A model that scores 100 % on its own test set can still be useless in deployment, because it has fit the exact shape of its training inputs, not the concept behind them.**

How it was found — by elimination, not by guessing

Test run	Result	Conclusion
Original training data through the full system	100 %	The system / fusion code is fine
Fault (OCC) data through the model	92 %	The model itself is fine
Engine Off data with no transitions	0 %	Transitions are not the cause

By elimination, only one thing remained: the difference in **temporal structure** between the jumpy training data and the smooth realistic data. The fix was not cosmetic — the data had to be regenerated so its temporal structure matched reality, which meant cutting the original 30-second windows down to **4-second** windows and **retraining the router on data that included the smooth variations it had never seen**. Because the data representation changed at the root, every downstream model and the fusion logic had to be rebuilt on top of it. That rebuild is v2.0.

4.3 Four more failures the redesign had to absorb

- **A model with no signal to learn.** A behaviour model once scored 33 % — pure chance for a 3-class problem. The cause was not the model but a **broken generator** producing the same Δ RPM for every class: there was literally no learnable difference in the data. *Garbage in, garbage out*, made concrete — a bad score is a question, not a verdict.
- **A “fix” that destroyed a working model.** Switching to stride-1 sliding windows — a reasonable-looking patch for a streaming problem — silently created windows containing **multiple states at once**, and a model with one label per window collapsed from 99.9 % to 67 %. Fighting the architecture to fit the data is the signal that the **data representation** is wrong, not the model.
- **“Bad accuracy” was sometimes a data bug.** More than once a poor number was a contaminated dataset — a final-test set silently mixed with mislabelled sequences, where *the models were being punished for being right because the ground truth was wrong*. The discipline: **when the numbers do not make sense, suspect the data before blaming the model**.
- **A pipeline can be worse than its parts.** At one point the fused system scored 80 % while its load model alone scored 98.6 % on the same data — the router upstream was mis-routing a fraction of inputs. **Integration, not the individual components, is where a multi-model system fails.**

4.4 The two decisive discoveries about the data

Normalization was the single biggest unlock. For a long time a behaviour model was stuck, unable to learn. The moment its inputs were **normalized**, it jumped to 99 %. The reason is scale: RPM sits near 2000, temperature near 100, vibration near 0.01. Without normalization the large-magnitude variables dominate the loss and the network effectively ignores the small ones — so vibration and pressure, which carry real information, were being drowned out by RPM. This is why **every neural model in the final system normalizes its inputs internally**, and it is one of the clearest demonstrations of the thesis: nothing about the *model* changed, only the *preparation of the data*, and that alone was the difference between not learning and 99 %. A related choice followed — I refused to normalize by dividing each variable by an arbitrary constant (RPM/10000, temperature/100), because hard-coded divisors assume fixed limits the data may not respect; the correct approach *learns* each variable's scale from the data itself (Keras `Normalization().adapt()`).

The cost of a hint, quantified. Given only normalized values, the behaviour model reached ~91 %. Given one engineered feature — the pre-computed ΔRPM — it reached ~99.6 %. That ~8-point gap is the thesis expressed as a number: “the model figures it out on its own” (91 %) versus “I tell it exactly what to look for” (99.6 %). This measurement is why ΔRPM was ultimately kept in the final behaviour model — the one deliberate exception (Section 5.3).

4.5 Why the LSTM was dropped

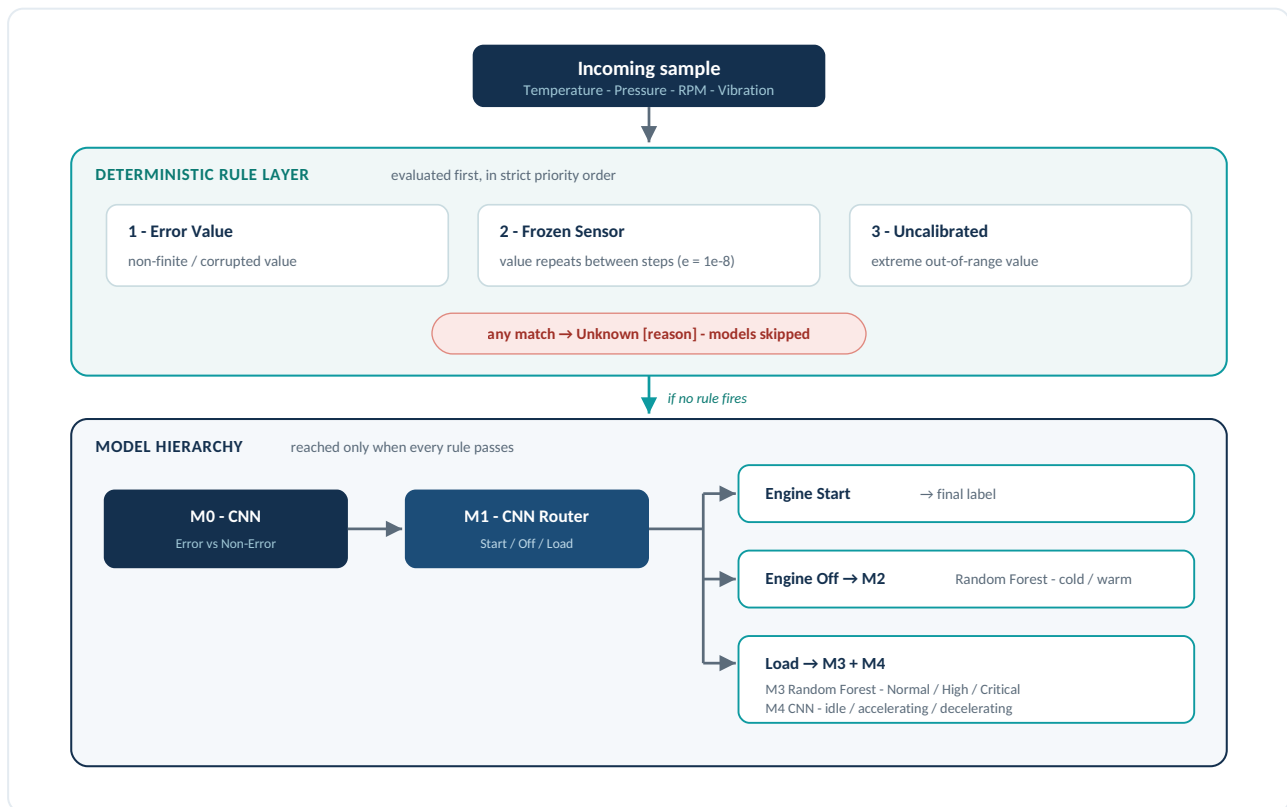
An earlier version used an LSTM for behaviour. It was removed in v2.0 for a specific reason: an LSTM's strength is *long-range* temporal memory, but behaviour is defined over a 2-second window. Reading a short, local pattern in a fixed window is exactly what a 1-D CNN does — faster, fewer parameters, less prone to overfitting on short contexts. The LSTM was not removed because it failed; it was removed because it was the **wrong tool for a short-context problem**. Knowing when a tool is overkill is part of the engineering.

5 The Final Architecture

The final system replaces “one model for everything” with a hierarchy in which each stage handles the task it is suited for. The controlling principle, earned through Section 4, is a division of labour: **deterministic faults are handled by rules; learnable states are handled by models**.

5.1 Decision flow

Every incoming sample is checked against a rule layer *first*, in strict priority order — **Error Value > Frozen Sensor > Uncalibrated > Models** — and only reaches the models if no rule fires:



The rule layer runs on a causal 4-step window (the previous three samples plus the current one). The frozen-sensor rule fires when temperature, pressure, or vibration repeats between consecutive steps (within $\epsilon = 10^{-8}$), or when RPM repeats while non-zero.

5.2 The models

Model	Task	Type	Input	Key detail
M0	Error vs Non-Error	1-D CNN	(4, 4)	values + per-step deltas of all 4 variables → 8 features
M1	Router (Start / Off / Load)	1-D CNN	(4, 4)	same values + deltas design
M2	Engine Off (cold / warm)	Random Forest, 100 trees	(4,)	a snapshot problem
M3	Load (Normal / High / Critical)	Random Forest, 200 trees	(4,)	a snapshot problem
M4	Behaviour (idle / accel / decel)	1-D CNN	(2, 5)	input augmented with ΔRPM (§5.3)

The two convolutional routers (M0, M1) share an architecture: internal normalization, then per-step deltas of all four variables concatenated to the normalized values, then two **Conv1D** layers (64 then 128, kernel 2), flatten, dense, dropout, softmax. Computing the deltas *inside* the model rather than as external preprocessing keeps feature engineering part of the trained artifact. M4 is a lighter network (two **Conv1D** layers, global max-pooling, dense) suited to its 2-step window.

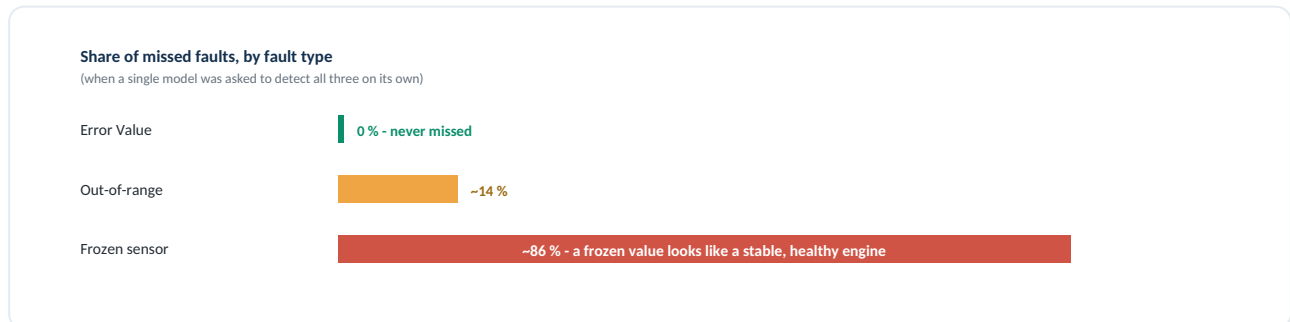
5.3 The one hint that stayed — stated plainly

The project's guiding rule was to withhold hints and let the models discover the patterns. It was tested constantly and held in every case but one, reported here rather than hidden. **M4 keeps an engineered feature: ΔRPM .** Its input is augmented from (2, 4) to (2, 5) by appending the change in RPM between the two steps. As Section 4.4 quantified, this feature is worth ~8 accuracy points — and it was kept, deliberately, in the single model where the class is literally the variation in RPM. The

tension between purity and pragmatism was resolved on the side of pragmatism, once, and knowingly. Every other model receives only normalized values and computes its own internal deltas; none is handed a pre-solved answer.

5.4 Why some faults are rules, not models — the sharpest finding

During development, the router model was asked to detect the three fault types on its own. Its competence was radically uneven:



The pattern is the thesis in its sharpest form. The **numerically extreme** error — a wildly impossible value — is caught perfectly, because it is unmistakable. The **contextually subtle** error — a sensor stuck at a plausible constant — is missed almost every time, because a frozen value *looks like* a stable, healthy engine. And a stuck sensor is precisely the fault a human diagnoses at a glance.

The model nails the error that a one-line `if` would also catch, and fails the error that actually requires understanding the context.

This is not a flaw to be trained away — it is the finding, and it directly produced the architecture in Section 5.1. **Because** the model cannot reliably detect a frozen or corrupted value, those faults are handled by deterministic rules given *priority above the model*. A frozen-sensor rule never misses a frozen sensor; the model missed 86 % of them. The two-tier uncalibrated design (Section 3) is the same principle one level deeper: the rule owns the extreme zone, and the model is trained to own the moderate zone the rule cannot reach. **The system reaches 99.8 % end-to-end precisely because it does not ask the model to do the thing the model is bad at.** This is what “hybrid” means in the title — not that ML and rules coexist, but that each was given exactly the part of the problem it handles better than the other.

6 Results

6.1 Per-model accuracy

Each model was evaluated on **two independent held-out sets** (a test set and a separate final-test set) — a stronger check than a single split. Bars show final-test accuracy:



Model	Task	Test	Final Test
M0	Error vs Non-Error	99.96 %	99.97 %
M1	Router (Start / Off / Load)	100.0 %	99.99 %
M2	Engine Off (cold / warm)	100.0 %	100.0 %
M3	Load (Normal / High / Critical)	100.0 %	100.0 %
M4	Behaviour (idle / accel / decel)	99.70 %	99.55 %

6.2 End-to-end system

99.8 %

the complete pipeline — rules plus all five models, running sample-by-sample as in deployment — benchmarked against a **164,920-row** labelled dataset built as a realistic sequence of engine life: off → start → load cycles, with faults injected throughout.

The remaining errors are not random. **Nearly all sit exactly on the boundary between adjacent states** — Normal versus High Load, or idle versus decelerating — where the defining values differ by less than one unit and a window can genuinely belong to either class. This is a property of how the states are *defined*, not a model failure: a boundary is ambiguous by construction. Just as importantly, the system makes **no serious errors** — it never confuses Normal Load with Critical Load, and never raises a false fault alarm on a healthy engine.

REPRODUCIBILITY & TRAINING RIGOUR

Fixed random seeds throughout (42 / 43) · internal `Normalization().adapt()` in every neural model · EarlyStopping with best-weights restore + ReduceLROnPlateau · stratified 70 / 15 / 15 split for M4 · automatic inverse-frequency class weights for M0 · every model evaluated on two independent held-out sets · the end-to-end benchmark is separate from all per-model test sets.

7 The Running System — From Models to Product

The classifier is not a notebook of models — it runs as a product, with a control panel, two operating modes, and an alert policy. This layer is where several of the journey's hardest bugs were finally closed.

7.1 Operating modes

- **LIVE mode.** A sensor simulator streams samples continuously (~2 per second); the core classifies each one on arrival and publishes the current state, the decision route through the hierarchy (e.g. `M0 → M1 → M3+M4`), and the live context window to `latest.json`, which the UI polls every second.
- **TEST mode.** The core runs the full 164,920-row labelled benchmark end-to-end and writes a row-by-row evaluation CSV — sensor values, predicted label, true label, and the route taken — with a progress bar and ETA in the control panel. Everything in Section 6.2 comes from this mode.

A single control panel (System **OFF / ON**, Mode **TEST / LIVE**) is the only entry point; when the system is off, nothing runs.

7.2 Alert policy — domain knowledge in the alarms too

Unknown — sensor malfunction

High Load sustained

Critical — immediate

Any Unknown raises a yellow warning (treat the *sensor*, not the engine, as the suspect). High Load only escalates to amber when *sustained* — a moment of high load is normal driving. Critical is red immediately. And one detail encodes engine sense directly into the alarm logic: **a detected deceleration cancels high/critical warnings** — an engine that is slowing down is resolving the condition, so the system does not keep shouting about a problem that is already ending.

7.3 Two runtime guards that closed old bugs

- **Forced warm-up.** The first four samples (before the context window fills) are labelled by a temperature rule that is *identical* to the benchmark's own labeller — so the system and its ground truth can never disagree about the warm-up period.
- **Engine-reset detection.** Whenever RPM drops from positive to zero, the context window is cleared. This is the final, permanent fix to the “**windows crossing boundaries**” problem that recurred through the whole project — first as training windows straddling unrelated blocks, later as false RPM spikes when one test sequence bled into the next. The cure was never a smarter model; it was making the *context* honest.

8

What the Project Revealed About the Limits of ML

The high accuracy is the least interesting result. These are the findings the project was built to produce:

- 1 **AI is good at the obvious error and bad at the subtle one.** Perfect detection of an impossible value; 86 % miss rate on a frozen sensor a human spots instantly. Competence scales with how *distinct* the pattern is in the data, not with how *hard* the problem looks to us.
- 2 **A model fits the shape of its data, not the concept.** Identical value ranges with a different temporal structure took the system from 100 % per-model to 1 % end-to-end. Validation accuracy and deployment accuracy are different quantities.
- 3 **Integration is where systems fail, not components.** Every model at ~100 %, the assembled system at 80 %, because of routing between them.
- 4 **High accuracy can be a lie.** Several bad scores were data bugs, not model failures — including ground truth that was itself wrong. An accuracy figure is only as trustworthy as the labels behind it.
- 5 **Data preparation beats model choice.** Normalization alone was the difference between a model that could not learn and one at 99 % — same architecture, better-prepared data.
- 6 **Rules do not err; models do.** Where a deterministic rule is reliable, it is simply better — and part of engineering an ML system well is knowing which parts of the problem should not be machine learning at all.

9

Limitations and Future Work

- **Synthetic data.** The system was trained and evaluated entirely on Python-generated data with added noise. Real-sensor validation is the clear next step and the most likely source of surprises.
- **Schema and unit errors are not covered.** The system detects *physically impossible* values, but cannot detect a sensor reporting a plausible value in the *wrong unit* (°F where °C is expected, or a swapped column) — the value passes every range check while being completely wrong.

- **Boundary ambiguity.** The residual error is concentrated on class boundaries, inherent to hard-thresholded state definitions. A system that must act on near-boundary states might model the boundaries probabilistically rather than as hard cuts.

CLOSING

The system works — 99.8 % on 164,920 rows, with no dangerous confusions and no false alarms. But the result worth defending is the one underneath the number: reaching it required treating the machine learning as the **last** step, not the first. The models only became reliable once the problem was understood physically, the data was structured to match reality, and each task was handed to the tool — rule, forest, or network — actually suited to it. The failures documented here were not detours around that lesson. They were the lesson.

— References

I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

The *Clever Hans effect* in machine learning — models that appear to solve a task while relying on a spurious correlation in the data.

A. Ng. *Deep Learning Specialization*, DeepLearning.AI.